

Question 1

- Dynamic programming is an effective technique for solving problems that exhibit overlapping subproblems. In the case of $f(n, k)$, the function exhibits overlapping subproblems because the computation of $f(n, k)$ depends on the values of $f(n-1, k-1)$ and $f(n-1, k+1)$. By storing the results of previously computed subproblems, we can avoid recomputing them multiple times, which can significantly improve the efficiency of the algorithm.
- A 2D array is a good choice for the dynamic programming data structure for this problem. The array can be indexed by n and k , where n represents the number of elements and k represents the difference between n and k . This allows us to store the precomputed values of $f(n, k)$ for all possible values of n and k .
- Here's a sketch of the data structure (2D array) you would use to compute $f(6, 3)$:

	0	1	2	3	4	5
1	B	B	B	B	B	B
2	B					
3	B					
4	B					
5	B					
6	B					

The cells with a value represent the precomputed values of $f(n, k)$. The empty cells represent values that haven't been computed yet. The cell $(6, 3)$ is the target value we want to compute.

- In the below data structure, the first row ($n = 1$) would have 'B' in all cells. This indicates that the values in this row are simply the values of k .

	0	1	2	3	4	5
1	B	B	B	B	B	B
2	B	B				
3	B					
4	B					
5	B					
6	B					

- The arrows from the cell $(6, 3)$ would point to the cells $(5, 2)$ and $(5, 4)$. These represent the two recursive calls made by $f(6, 3)$:
 - $f(6, 3) = f(5, 2) + f(5, 4)$
- A valid iteration order for a dynamic programming algorithm to compute $f(n, k)$ would involve filling the matrix row by row, starting from the base cases and moving towards the target cell $f(n, k)$. This can be achieved using nested loops, either in the top-down (recursive) or bottom-up (iterative) approach.
- Here's the complete pseudocode for an iterative dynamic programming algorithm to compute $f(n, k)$:

```
function dynamic_f(n, k):
```

```
    # Create a 2D array to store precomputed values
```

```
table = new array(n + 1, k + 1)
```

```
# Fill in the base cases (first row and first column)
```

```
for I in range(1, n + 1):
```

```
    table[i][0] = I # f(n, 0) = n (base case)
```

```
for j in range(1, k + 1):
```

```
    table[1][j] = j # f(1, k) = k (base case)
```

```
# Fill the remaining entries using the recurrence relation
```

```
for I in range(2, n + 1):
```

```
    for j in range(1, min(I, k + 1)): # Limit j to min(I, k+1) to avoid out-of-bounds access
```

```
        table[i][j] = table[I - 1][j - 1] + table[I - 1][j + 1]
```

```
# Return the target value
```

```
return table[n][k]
```

Explanation:

1. Initialization:

- We create a 2D array table to store the precomputed values of $f(n, k)$. The size of the array is $(n + 1) \times (k + 1)$ to accommodate for indexing from 1.

2. Base Cases:

- We iterate through the first row ($n = 1$) and the first column ($k = 0$) and fill them according to the base cases:
 - $table[i][0] = i$ for $1 \leq i \leq n$ ($f(n, 0) = n$)
 - $table[1][j] = j$ for $1 \leq j \leq k$ ($f(1, k) = k$)

3. Dynamic Programming Loop:

- We use nested loops to iterate through the remaining entries of the table in a bottom-up manner:
 - The outer loop iterates over n from 2 to N (skipping the first row filled with base cases).
 - The inner loop iterates over j from 1 to a minimum of I and $k + 1$. This limitation prevents accessing out-of-bounds elements in the table when k is close to n .
- For each element $table[i][j]$, we calculate its value using the recurrence relation: $table[i][j] = table[I - 1][j - 1] + table[I - 1][j + 1]$

4. Return Value:

- Finally, we return the value stored in $table[n][k]$, which represents the target value $f(n, k)$.

h. Time Complexity:

- The time complexity of the dynamic programming solution is $O(n * k)$, where n and k are the input parameters.
- This is because the nested loops iterate a total of $n * k$ times, filling in the entries of the table.

Space Complexity:

- The space complexity is $O(n * k)$ as well.
- This is because the table array stores the precomputed values of $f(n, k)$ for all possible combinations of n and k within the specified range.

The space complexity of the algorithm can be reduced by using only two rows of the matrix at a time instead of the entire matrix. Since each row only depends on the previous row, we can optimize the space complexity to $O(k)$ instead of $O(n*k)$ by maintaining only two rows in memory and updating them iteratively.

Question 2

Explanation:

1. The function takes two matrices a and b as input. It initializes variables for m (size of a), n (size of b), current alignment position (i, j) , and the maximum score found so far.
2. It iterates through each element of a .
3. Within the loop, it iterates through all possible positions in b where a could be placed (considering boundaries).
4. For each potential alignment, it calculates the score by multiplying the corresponding elements of a and b .
5. It compares the current score with the `max_score`. If higher, it updates the `max_score` and the corresponding alignment position (i, j) .
6. After iterating through all elements of A , the function returns the best alignment (i, j) and the maximum score.

```
function GreedyMatrixAlignment(a, b):
    # Initialize variables
    m = len(a) # size of matrix a
    n = len(b) # size of matrix b
    i, j = 0, 0 # current alignment position
    max_score = 0 # stores the best score found so far

    # Loop through elements of a
    for a_row in range(m):
        for a_col in range(m):
            # Calculate potential score for each possible alignment in b
            for b_row in range(n):
                for b_col in range(n):
                    # Check if alignment is valid (within b's boundaries)
                    if (b_row + a_row < n) and (b_col + a_col < n):
                        # Calculate score for this alignment
                        current_score = a[a_row][a_col] * b[b_row][b_col]

                        # Update best alignment if current score is higher
                        if current_score > max_score:
                            max_score = current_score
                            i, j = b_row, b_col

    # Return the best alignment with maximum score
    return i, j, max_score
```